Last Minute Angular Interview Preparation - Complete Q&A Bank

This document contains automatically generated, deduplicated, categorized, and numbered answers to your interview questions.

1. Basic Angular Questions

Q1. What is Angular?
Angular is a TypeScript-based open-source web framework developed by Google used to build dynamic single-page applications (SPAs).

Q2. What are the main features of Angular?
Key features include Component-based architecture, Dependency Injection, Directives, Two-way data binding, Routing, RxJS integration, and Forms handling.

Q3. Describe the Angular application architecture.
Angular architecture is based on Modules (NgModules). It consists of Components (UI), Services (business logic), Templates (HTML views), Directives, and Dependency Injection to bind everything together.

Q4. How an Angular App gets Loaded and Started?
The browser loads `index.html`, which triggers `main.ts`. `main.ts` bootstraps the root module (`AppModule`). `AppModule` then bootstraps the root component (`AppComponent`), rendering it inside the `<app-root>` tag.

Q5. What is a Bootstrapped Module & Bootstrapped Component?
A bootstrapped module (typically `AppModule`) is the entry point of the application. A bootstrapped component (typically `AppComponent`) is the root view inserted into the `index.html` file.

Q6. What is a component in Angular, and how is it used?
Components are UI building blocks in Angular, defined by a TypeScript class (logic), HTML template (view), and CSS styles. They are created using the @Component decorator.

Q7. What is a Selector and Template?
A selector (e.g., `app-root`) is the custom HTML tag used to instantiate a component. A template defines the HTML view associated with that component.

Q8. What are Parent-Child Components?
A parent component is one that contains another component within its template. The contained component is the child. This structure builds the component tree.

Q9. What is a module in Angular, and what is its purpose?
A module is a container that organizes an application into cohesive blocks. It uses the @NgModule decorator to declare components, directives, pipes, and provide services.

Q10. What are Angular directives? Name a few commonly used ones.
Directives are classes that modify the DOM. Types include Structural (*ngIf, *ngFor) which change layout, and Attribute (ngClass, ngStyle) which change appearance or behavior.

Q11. What is data binding in Angular? Explain different types.
Data binding synchronizes data between the model and the view. Types include String Interpolation ({{}}), Property Binding ([]), Event Binding (()), and Two-way Binding ([()]).

Q12. What is String Interpolation in Angular?
A form of one-way data binding that embeds expressions into marked-up text using double curly braces: `{{ expression }}`. It evaluates the expression and converts it to a string.

Q13. What is Property Binding in Angular?
One-way data binding from the component to a DOM property, written with square brackets: `[property]='value'`. It passes values (including booleans and objects) to HTML elements.

Q14. What is Event Binding in Angular?
One-way data binding from the view to the component, written with parentheses: `(click)='myFunction()'`. It listens to user actions and triggers component methods.

Q15. What is two-way binding, and how do you implement it in Angular?
Two-way binding syncs data bidirectionally between the component and the view. Implemented using the 'banana-in-a-box' syntax: `[(ngModel)]='property'`. Requires `FormsModule`.

Q16. What is Angular CLI, and what can it be used for?
The Angular CLI is a command-line interface tool used to initialize, develop, scaffold (generate components/services), serve, and build Angular applications.

Q17. What is NPM?
Node Package Manager (NPM) is a repository and command-line utility used to install, share, and manage project dependencies (like Angular itself) in JavaScript/TypeScript environments.

Q18. Where to store static files in an Angular project?
Static files like images, fonts, and icons should be stored in the `src/assets` folder. This folder is automatically copied to the output directory during the build process.

Q19. What is the role of the angular.json file in Angular?
It is the workspace configuration file. It controls build configurations, project settings, paths to assets/styles/scripts, and CLI defaults.

Q20. What is Dependency Injection in Angular?
A design pattern where a class receives its dependencies from external sources rather than creating them itself. Angular's DI framework instantiates and provides these objects.

Q21. What are Angular services, and why are they useful?
Services are classes containing business logic, API calls, or shared data. They are injected into components to keep UI code lean and promote reusability across the app.

Q22. What is a Singleton Service in Angular?
A service that is instantiated only once per application lifecycle (typically provided in `'root'`). All components injecting it share the exact same instance and state.

Q23. What are lifecycle hooks in Angular?
Interfaces that allow developers to tap into specific moments in a component's lifecycle, from creation (`ngOnInit`) to destruction (`ngOnDestroy`).

Q24. What is a Constructor in Angular?
A default TypeScript feature used to instantiate a class. In Angular, it should primarily be used for dependency injection, not heavy component initialization logic.

Q25. Explain ngOnInit and ngOnDestroy lifecycle.
`ngOnInit` runs once after the first `ngOnChanges`, used for component data initialization. `ngOnDestroy` runs just before the component is destroyed, used for cleanup (like unsubscribing).

Q26. What is the difference between constructor and ngOnInit?
The constructor is a TypeScript feature for class instantiation and dependency injection. `ngOnInit` is an Angular lifecycle hook called after Angular sets input properties, making it the right place for component initialization logic.

Q27. What is the purpose of ngAfterViewInit?
It is called after Angular has fully initialized a component's view and child views. It is useful for interacting with the DOM directly or reading values from `@ViewChild`.

Q28. What is ViewEncapsulation in Angular?
It determines how component styles are scoped. Modes include Emulated (default, scoped to component), None (global), and ShadowDom (uses browser native Shadow DOM).

Q29. How do you apply conditional styling in Angular components?
Using the `[ngClass]` or `[ngStyle]` directives. Example: `[ngClass]="{'active-class': isActive}"` or `[ngStyle]="{'color': isError ? 'red' : 'green'}"`.

Q30. How do you pass data between parent and child components?
From parent to child using `@Input()` properties. From child to parent using `@Output()` properties combined with `EventEmitter`.

Q31. What is @Input Decorator?
A decorator that marks a class field as an input property, allowing data to flow from a parent component into a child component.

Q32. What is @Output Decorator and Event Emitter?
A decorator used with `EventEmitter` to emit custom events from a child component to its parent component, facilitating child-to-parent communication.

Q33. What is the difference between AngularJS and Angular?
AngularJS (Angular 1.x) is JavaScript-based and uses controllers/$scope. Angular (2+) is TypeScript-based, uses a component-based architecture, and is significantly faster.

Q34. What is a template-driven form in Angular?
Forms where logic is primarily handled in the HTML template using directives like `ngModel`. Best for simple, static forms.

Q35. What is a reactive form in Angular?
Forms built programmatically in the component TypeScript class using `FormGroup` and `FormControl`. Best for complex, dynamic, and highly testable forms.

Q36. What is the difference between template-driven and reactive forms?
Template-driven is asynchronous, template-heavy, and relies on two-way binding. Reactive is synchronous, class-heavy, immutable, and relies on observables.

Q37. What is FormGroup in Angular Forms?
A collection of `FormControl` instances. It tracks the collective value and validity state of a group of controls.

Q38. What is FormControl in Angular?
A class that tracks the value and validation status of an individual form input element.

Q39. What is FormBuilder in Angular?
A helper service that provides syntactic sugar (shorthand methods) for generating `FormGroup`, `FormControl`, and `FormArray` instances quickly.

Q40. How do you validate forms in Angular?
By applying built-in validators (like `Validators.required`) or creating custom validator functions, then checking the control's `.invalid` or `.hasError()` state to show UI feedback.

Q41. What is HttpClient in Angular?
A built-in service in `@angular/common/http` used to communicate with backend APIs over HTTP, relying entirely on RxJS Observables to handle requests and responses.

Q42. How do you make an API call using HttpClient?
Inject `HttpClient`, call a method like `this.http.get(url)`, and call `.subscribe()` on the returned Observable to execute the request and handle the data.

2. Intermediate Angular Questions

Q1. What is RxJS in Angular?
Reactive Extensions for JavaScript (RxJS) is a library for reactive programming using Observables to handle asynchronous operations and streams of data.

Q2. What are Asynchronous operations?
Operations that run independently of the main program flow (e.g., API calls, timers) allowing the app to remain responsive while waiting for the operation to complete.

Q3. What are Observables and Promises?
Both handle async operations. Promises resolve/reject a single value once and execute immediately. Observables handle multiple values over time, are lazy (don't execute until subscribed), and can be canceled.

Q4. What is the difference between Promise and Observable?
Observables emit multiple values, are lazy, and can be cancelled using `unsubscribe()`. Promises emit a single value, execute immediately, and cannot be cancelled.

Q5. What is the difference between components, structural, and attribute directives?
Components are directives with templates. Structural directives (*ngIf, *ngFor) alter DOM layout by adding/removing elements. Attribute directives (ngClass, ngStyle) alter the appearance/behavior of existing elements.

Q6. What are pipes in Angular?
Functions used directly in HTML templates to transform data for display purposes (e.g., formatting dates, currencies). Syntax: `{{ value | pipeName }}`.

Q7. How do you create a custom pipe in Angular?
Create a class decorated with `@Pipe`, implement the `PipeTransform` interface, and write the transformation logic inside the `transform()` method.

Q8. What is Chaining Pipes?
Applying multiple pipes to a single expression sequentially. The output of the first pipe becomes the input for the second. Example: `{{ today | date | uppercase }}`.

Q9. What is the purpose of the async pipe?
It automatically subscribes to an Observable or Promise directly in the HTML template, returns the latest value, and automatically unsubscribes when the component is destroyed to prevent memory leaks.

Q10. What is Change Detection in Angular?
The mechanism by which Angular checks for changes in the application state and synchronizes those changes with the DOM.

Q11. What is the difference between OnPush and Default Change Detection?
Default checks every component in the tree on any async event. OnPush only checks the component if its `@Input` reference changes, an event originates from it, or if it's manually triggered.

Q12. What are interceptors in Angular?
Classes implementing `HttpInterceptor` used to inspect, transform, or modify outgoing HTTP requests (e.g., adding Auth tokens) and incoming responses globally.

Q13. What is the purpose of catchError in RxJS?
An operator used within a `.pipe()` chain to catch errors emitted by the source observable, handle them (e.g., logging), and gracefully return a safe fallback observable or rethrow the error.

Q14. How does Angular handle routing?
Through the `RouterModule`. Developers define an array of `Routes` matching URL paths to components, which are rendered dynamically inside the `<router-outlet>` tag.

Q15. What is router outlet?
A directive (`<router-outlet>`) acting as a placeholder in a template where the Angular router dynamically inserts the component matched by the current URL.

Q16. What are router links?
An attribute directive (`[routerLink]`) used in templates to navigate between routes declaratively without triggering a full page reload.

Q17. What is a Router Guard in Angular?
Interfaces (like `CanActivate` or `CanDeactivate`) that act as middleware. They run before navigation completes, allowing you to grant or deny access based on conditions like authentication.

Q18. What are query parameters in Angular routing?
Optional parameters appended to the URL (e.g., `?page=1&sort=asc`) used to pass non-essential state. They are accessed via the `ActivatedRoute` service.

Q19. What is ActivatedRoute in Angular?
A service that provides access to information about the currently active route, including URL parameters, query parameters, and route data.

Q20. What is lazy loading in Angular?
A performance optimization technique where NgModules (and their components) are only downloaded and loaded when the user navigates to their specific routes, reducing the initial bundle size.

Q21. How do you dynamically create and load components in Angular?
Using `ViewContainerRef.createComponent()` to instantiate a component programmatically and insert it into the DOM at runtime.

Q22. What is Decorator? What are the types of Decorator?
Decorators are functions that attach metadata to classes, properties, or methods. Types include Class (`@Component`), Property (`@Input`), Method (`@HostListener`), and Parameter (`@Inject`).

Q23. What is Provider in Angular?
An instruction to the Dependency Injection system detailing how to obtain or create a dependency (like a service). Providers can be registered in components, modules, or globally.

Q24. What is the role of @Injectable Decorator in a Service?
It marks a class as available to be injected as a dependency. The `{ providedIn: 'root' }` syntax inside it allows Angular to optimize it as a global singleton.

Q25. What is AOT (Ahead-of-Time) compilation and how does it differ from JIT?
AOT compiles Angular HTML and TypeScript code into efficient JavaScript during the build phase, before the browser downloads it. JIT (Just-in-Time) compiles it in the browser at runtime. AOT results in faster rendering and smaller bundles.

Q26. What is Ivy in Angular?
The default modern compilation and rendering pipeline in Angular (since v9). It drastically reduces bundle sizes, speeds up compilation, and makes debugging easier.

Q27. What is differential loading in Angular?
A build process that creates two separate JavaScript bundles: a modern one for newer browsers (ES2015+) and a legacy one (ES5). The browser downloads only the bundle it supports.

Q28. What are the various ways to communicate between the components?
Using `@Input()`/`@Output()`, Template Reference Variables, `@ViewChild()`, `<ng-content>` (projection), and shared Services (using RxJS Subjects).

Q29. What is Template Reference Variable in Angular?
A variable defined in a template using a hash symbol (e.g., `#myInput`) that creates a direct reference to a DOM element, component, or directive, usable anywhere within that template.

Q30. What is the role of ViewChild in Angular?
A property decorator that allows a parent component to query and access a child component, directive, or DOM element reference directly from its TypeScript class.

Q31. How to access the child component from parent component with ViewChild?
Decorate a property with `@ViewChild(ChildComponent) child!: ChildComponent;`. The child component instance becomes available to the parent inside the `ngAfterViewInit` lifecycle hook.

3. Advanced Angular Questions

Q1. What is state management in Angular?
The architectural process of managing the data (state) of an application across various components, often handled by RxJS `BehaviorSubject` in services or libraries like NgRx.

Q2. What is NgRx in Angular?
A suite of reactive libraries for Angular that implements the Redux pattern, utilizing RxJS to manage global and local application state predictably.

Q3. What are Actions, Reducers, and Effects in NgRx?
Actions are unique events describing state changes. Reducers are pure functions that take the current state and an action to produce a new state. Effects isolate side effects (like HTTP calls) from components.

Q4. What is a Store in NgRx?
A single, controlled, immutable state container that holds the application state. Components dispatch actions to it and select data from it using selectors.

Q5. How does Angular handle Dependency Injection hierarchy?
DI in Angular is hierarchical. A dependency can be provided at the App/Root level (singleton), Module level, or Component level (creating a new instance scoped to that component and its children).

Q6. What is Hierarchical Dependency Injection?
The concept that if a service is provided at a specific component level, a new instance is created for that component and its children, isolating its state from the rest of the app.

Q7. What are multi-providers in Angular?
Providers configured with `{ multi: true }`. This allows multiple values or classes to be provided under a single injection token, which is heavily used for providing multiple HTTP Interceptors.

Q8. What is forwardRef in Angular?
A utility function used to refer to a class before it is defined. It is commonly needed to resolve circular dependencies in DI or when providing `NG_VALUE_ACCESSOR` for custom form controls.

Q9. What is Angular Universal?
A technology that renders Angular applications on the server (Server-Side Rendering) before sending the fully generated HTML to the client browser.

Q10. What is Server-Side Rendering (SSR)?
The process of generating the full HTML of a page on the server in response to navigation. This improves SEO, social media scraping, and initial page load performance.

Q11. How does Angular handle SSR with Angular Universal?
It executes the Angular app on a Node.js server, serializes the application state, and sends raw HTML to the browser. Once loaded, the client app 'bootstraps' over the server HTML (hydration).

Q12. What is prerendering in Angular Universal?
A build-time process that runs the Angular app to generate static HTML files for specific routes. These static files can be served by any web server, resulting in extremely fast loads.

Q13. How do you optimize Angular applications for performance and scalability?
Use Lazy Loading, OnPush change detection, AOT compilation, `trackBy` in `*ngFor`, pure pipes instead of methods in templates, tree shaking, and robust state management like NgRx.

Q14. What are Web Components, and how can Angular elements be used as Web Components?
Web Components are standard-based, encapsulated custom HTML elements. The `@angular/elements` package allows packaging standard Angular components as native Custom Elements usable in any web framework.

Q15. What is the Shadow DOM, and how does Angular use it?
A web standard that encapsulates HTML markup and CSS. Angular utilizes it via `ViewEncapsulation.ShadowDom` to strictly isolate a component's styles from the global application scope.

Q16. What is Content Projection? What is <ng-content>?
Content Projection is a way to pass HTML content from a parent component into a child component's template. `<ng-content>` is the placeholder tag inside the child where the projected HTML renders.

Q17. What is the difference between ViewChild and ViewChildren? What is QueryList?
`@ViewChild` targets a single element/component. `@ViewChildren` targets multiple elements and returns a `QueryList`, which is an iterable object that updates automatically when the DOM changes.

Q18. What is ContentChild?
A decorator used to query and access the first element or directive that was projected into the component via `<ng-content>` from the parent.

Q19. What is the difference between ContentChild & ContentChildren?
`@ContentChild` accesses a single projected element. `@ContentChildren` accesses multiple projected elements and returns a `QueryList`.

Q20. Compare ng-Content, ViewChild, ViewChildren, ContentChild & ContentChildren?
`<ng-content>` projects HTML. `ViewChild/Children` query elements *within* the component's own template. `ContentChild/Children` query elements projected *into* the component from a parent.

Q21. What is an Angular Resolver?
A specialized route guard (implementing the `Resolve` interface) used to pre-fetch critical data before a route is fully activated, ensuring the component has its data the moment it renders.

Q22. How do you handle error messages globally in Angular?
By implementing a custom class extending `ErrorHandler` to catch unhandled client-side exceptions, and using `HttpInterceptors` to catch, format, and display server-side HTTP errors.

Q23. How does Angular handle memory leaks?
Memory leaks usually stem from unclosed Observable subscriptions or detached DOM nodes. Developers must clean up by calling `.unsubscribe()` in `ngOnDestroy` or relying on the `async` pipe.

Q24. What is the purpose of the Renderer2 service?
It provides a safe API to manipulate the DOM (e.g., `setStyle`, `addClass`) independently of the browser environment, which is crucial for Server-Side Rendering or running Angular in Web Workers.

Q25. How do you secure an Angular application?
Use Route Guards to restrict access, sanitize HTML/URL inputs to prevent XSS (Angular does this natively), implement CSRF protections using interceptors, and manage JWT tokens securely.

4. Scenario-Based & Security Questions

Q1. How do you improve performance in a large Angular application?
Implement route lazy loading, use `OnPush` change detection, decouple heavy logic into Web Workers, use pure pipes for transformations, use `trackBy` in loops, and avoid function calls in templates.

Q2. How would you handle large datasets in an Angular application?
Use virtual scrolling (via Angular CDK) to render only visible items, implement server-side pagination/filtering, and use efficient state management to avoid redundant API calls.

Q3. How do you debug an Angular application effectively?
Use Chrome DevTools, install the Angular DevTools extension to profile component trees/change detection, utilize strict mode, use `console.log/debugger`, and monitor the Network tab.

Q4. How would you implement internationalization (i18n) in Angular?
Use the built-in `@angular/localize` package, mark text with `i18n` attributes in templates, extract translation files using the CLI, and build separate localized bundles for each language.

Q5. What is Authentication & Authorization in Angular?
Authentication verifies *who* the user is (e.g., verifying login credentials). Authorization verifies *what* the user is allowed to do (e.g., checking roles to allow access to an admin route).

Q6. What is JWT Token Authentication in Angular?
A mechanism where the backend issues a JSON Web Token upon successful login. The Angular app stores this token and attaches it to subsequent HTTP headers to prove the user's authenticated identity.

Q7. What are the parts of JWT Token?
A JWT consists of three base64-encoded parts separated by dots: Header (algorithm type), Payload (user data/claims), and Signature (used to verify the token hasn't been tampered with).

Q8. How to implement the Authentication with JWT in Angular?
Create a login form, post credentials via `HttpClient`, receive the JWT, store it (e.g., localStorage/sessionStorage), and use an `HttpInterceptor` to attach it to all outbound API requests.

Q9. Which part of the request has the token stored when sending to API?
The token is strictly stored in the HTTP Headers of the request, typically under the `Authorization` header using the `Bearer <token>` format.

Q10. How to Mock or Fake an API for JWT Authentication?
Use tools like `json-server` for a quick mock backend, use Angular's `HttpTestingController` in unit tests, or intercept HTTP calls locally using an `HttpInterceptor` to return a mock JWT response.

Q11. How do you handle authentication in Angular?
By storing tokens securely, validating their expiration, dynamically showing/hiding UI elements based on authentication state (using a shared service), and protecting routes via Guards.

Q12. How do you prevent unauthorized access in Angular?
Implement `CanActivate` and `CanLoad` Route Guards that check a user's token validity and role assignments before permitting navigation to protected components or downloading lazy-loaded modules.

Q13. What is Auth Guard?
A specific implementation of an Angular Route Guard (like `CanActivate`) designed specifically to intercept routing and redirect unauthenticated users to a login page.

Q14. How to Retry automatically if there is an error response from API?
Use the `retry()` or `retryWhen()` operators from RxJS within the `.pipe()` chain of your `HttpClient` request or inside an `HttpInterceptor` to automatically retry the request a set number of times.

Q15. How do you handle file uploads in Angular?
Use an `<input type='file'>`, capture the file object on the `change` event, append it to a `FormData` object, and post it to the server using `HttpClient`.

Q16. How would you integrate Angular with a backend (Node.js, .NET, etc.)?
By designing RESTful endpoints on the backend and consuming them in Angular using `HttpClient` services, exchanging data formatted primarily as JSON.

Q17. What is your approach to handling errors in Angular services?
Pipe the HTTP request through the `catchError` RxJS operator, log the error locally, and use `throwError` to pass a user-friendly message back to the component or a global error handler.

Q18. How do you track and optimize performance metrics in Angular?
Run Google Lighthouse audits, use the Angular DevTools profiler to measure change detection cycles, and analyze bundle sizes using tools like `webpack-bundle-analyzer` or `source-map-explorer`.

5. TypeScript & OOPs Basics

Q1. What is Typescript? What are the advantages of Typescript over Javascript?
TypeScript is a strict syntactical superset of JavaScript that adds static typing. Advantages include compile-time error checking, robust IDE support/autocomplete, OOP features (interfaces/access modifiers), and easier refactoring.

Q2. What is the difference between let and var keyword?
`var` is function-scoped and allows hoisting (accessible before declaration). `let` is block-scoped (scoped to `{}` blocks) and prevents accessing the variable before it is declared.

Q3. What is Type annotation?
Explicitly specifying the type of a variable, parameter, or return value using a colon. Example: `let name: string = 'Angular';`.

Q4. What are Built in/ Primitive and User-Defined/ Non-primitive Types in Typescript?
Primitives are basic types like `string`, `number`, `boolean`, `null`, `undefined`. Non-primitives are complex, user-defined types like `classes`, `interfaces`, `enums`, and `arrays`.

Q5. What is "any" type in Typescript?
A type that completely disables TypeScript's type checking for that specific variable, allowing it to hold any value. It should be used sparingly as it defeats the purpose of TypeScript.

Q6. What is Enum type in Typescript?
A feature that allows developers to define a set of named constants (numeric or string-based), making code more readable. Example: `enum Direction { Up, Down, Left, Right }`.

Q7. What is the difference between void and never types in Typescript?
`void` indicates a function completes normally but returns no value. `never` indicates a function *never* completes (e.g., an infinite loop or a function that always throws an error).

Q8. What is Type Assertion in Typescript?
A way to override TypeScript's inferred type by telling the compiler 'trust me, I know what type this is.' Syntax involves angle brackets `<Type>value` or the `as` keyword `value as Type`.

Q9. What are Arrow Functions in Typescript?
A compact ES6 syntax for writing functions `() => {}` that lexically binds the `this` context, meaning `this` refers to the scope in which the arrow function was defined.

Q10. What is Object Oriented Programming in Typescript?
A programming paradigm based on the concept of 'objects' (containing data and methods). TypeScript enables robust OOP via classes, inheritance, encapsulation, and polymorphism.

Q11. What are Classes and Objects in Typescript?
A Class is a blueprint defining properties and methods. An Object is a specific instance created from that class blueprint using the `new` keyword.

Q12. What are Access Modifiers in Typescript?
Keywords controlling the visibility of class members. `public` (accessible anywhere), `private` (accessible only within the class), and `protected` (accessible within the class and its subclasses).

Q13. What is Encapsulation in Typescript?
The OOP principle of bundling data and methods that operate on that data within a single unit (class), using access modifiers to restrict direct external access to the internal state.

Q14. What is Inheritance in Typescript?
The OOP mechanism where a new class (child) derives properties and methods from an existing class (parent) using the `extends` keyword, promoting code reusability.

Q15. What is Polymorphism in Typescript?
The ability of different classes to be treated as instances of the same class through a common interface, or the ability of a child class to override a parent class's method with its own implementation.

Q16. What is Interface in Typescript?
A syntactical contract that defines the shape or structure of an object (what properties/methods it must have) without providing the implementation.

Q17. What's the difference between extends and implements in TypeScript?
`extends` is used by a class to inherit code and implementation from another class. `implements` is used by a class to guarantee it fulfills the structural contract defined by an interface.

Q18. Is Multiple Inheritance possible in Typescript?
TypeScript does not support multiple inheritance for classes (a class can only `extend` one parent class). However, a class can `implement` multiple interfaces.

6. Miscellaneous Angular Questions

Q1. What is the difference between ngModel and formControl?
`ngModel` is a template-driven directive used for two-way data binding on an input. `formControl` is an object used in reactive forms to programmatically track the value and validation status.

Q2. What is Tree Shaking in Angular?
A critical build-step optimization process performed by the bundler that detects and completely removes unused or dead code from the final JavaScript payload.

Q3. What is a Singleton pattern, and how does Angular use it?
A design pattern restricting a class to a single instance. Angular utilizes this heavily; services provided in `'root'` are instantiated exactly once and shared across the entire application.

Q4. What testing tools are used in Angular?
Jasmine (the testing framework for writing tests) and Karma (the test runner executing them in a browser) for unit tests. Protractor or newer tools like Cypress/Playwright are used for End-to-End (E2E) tests.

Q5. What is Jasmine and Karma?
Jasmine is a behavior-driven development framework providing syntaxes like `describe` and `it` to write assertions. Karma is the test runner that spawns a browser and executes the Jasmine tests.

Q6. How do you test services in Angular?
Set up a testing module using `TestBed`, inject the target service, and write assertion blocks to test its specific methods. Use spy objects to mock out its dependencies.

Q7. How to mock HTTP requests in unit tests?
Import the `HttpClientTestingModule` and inject `HttpTestingController`. This allows you to intercept outbound requests, provide mock JSON responses, and assert that the correct URLs were called.

Q8. What are E2E tests in Angular?
End-to-End tests simulate a real user navigating through the application in a browser, verifying that the entire stack (UI, Router, and Backend integration) functions correctly.

Q9. How do you debug memory leaks in Angular?
Inspect the Chrome DevTools Memory tab, take heap snapshots, locate un-garbage-collected components, and trace them back to unclosed RxJS subscriptions or manual DOM event listeners.

Q10. What is an Angular schematic?
A template-based code generator that supports complex logic. It is the underlying technology the Angular CLI uses to generate components, services, and perform framework migrations.

Q11. What is ng update in Angular?
A specific CLI command that safely updates your Angular workspace, its dependencies, and automatically executes migration scripts to rewrite code for breaking changes.

Q12. How do you create a PWA (Progressive Web App) with Angular?
By running `ng add @angular/pwa`. This command automatically sets up a Service Worker, a `manifest.webmanifest`, and configures the app for offline caching and home-screen installability.

Q13. What is ng-packagr in Angular?
A specialized build tool used by the Angular CLI to compile and package Angular libraries into the standard Angular Package Format (APF) so they can be published to NPM.

Q14. How to migrate from AngularJS to Angular?
Use the `@angular/upgrade` package to create a hybrid application running both frameworks simultaneously. Slowly rewrite components and services to the new framework, retiring the old ones one by one.

Q15. What is the purpose of tsconfig.json in Angular?
It is the primary TypeScript configuration file. It dictates compiler options like strictness, target JavaScript version (ES2015/ES2022), module resolution, and path mappings.

Q16. What are standalone components in Angular?
Introduced in Angular 14, standalone components are self-contained building blocks that directly import their own dependencies, removing the strict necessity of declaring them inside an `NgModule`.

Q17. What is Postman?
A popular graphical API client tool used by developers to construct, test, and document HTTP requests (GET, POST, etc.) and analyze the responses independent of the frontend application.